



哈爾濱工業大學(深圳)  
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

規格嚴格 功夫到家  
1920 — 2017

# Chapter 9

## Graphical User Interface Swing



# Content

- **Swing frame**
- **The principle of Swing graphic processing and drawing color**
- **Event mechanism**
- **Swing basic user components**
- **MVC pattern**



# JAVA GUI History - AWT

---

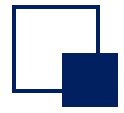
- ❑ When Java 1.0 first appeared, it included a library for basic GUI programming called the Abstract Window Toolkit (AWT).
- ❑ The basic AWT library delegates the task of handling user interface elements to the **native GUI toolkit** on each target platform, which is responsible for the creation and behavior of the user interface elements.
- ❑ Issue:
  - Writing high-quality, portable graphics libraries that rely on native user interface elements is difficult
  - There is no rich collection of user interface components
  - There are different bugs in the AWT user interface library on different platforms



# JAVA GUI History - Swing

---

- ❑ Sun and Netscape have created a user interface library called Swing.
- ❑ Swing, as an extension of Java 1.1, is now part of the Java 1.2 standard library.
- ❑ Swing is a toolkit (class library) for Java GUI programming (graphical interface design).
  - Swing is not a complete replacement for AWT, but is built on top of the AWT architecture.
  - Swing provides more powerful user interface components, such as drawn user interface classes.
  - AWT provides the underlying mechanisms of the window toolkit, such as event handling.
  - Swing uses pure Java code to emulate various controls, without using the native operating system's intrinsic methods, so Swing is cross-platform



# JAVA GUI History - JavaFX

---

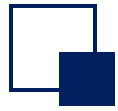
- ❑ When Swing was first released, users complained that it was too slow.
- ❑ And as computers get faster, users start complaining that Swing is too ugly.
- ❑ In 2007, JavaFX. It runs on a Java virtual machine and is optimized for animation and flashy effects.
- ❑ Starting with Java 11, JavaFX will no longer be packaged into Java.



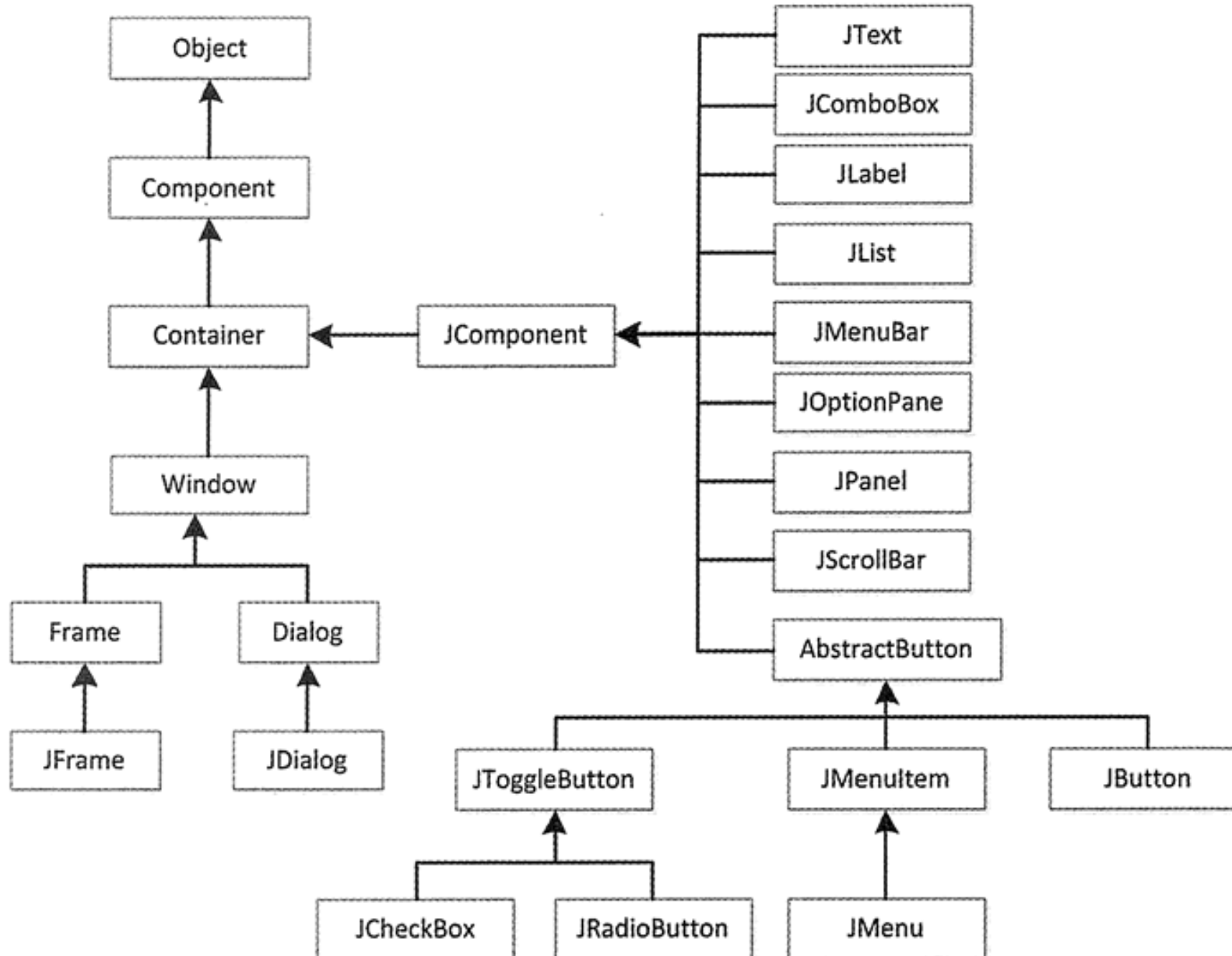
# Swing frame

---

- ❑ The Swing GUI consists of two elements: components and containers.
- ❑ A Java graphical interface made up of a variety of different types of "elements" such as windows, menu bars, dialog boxes, labels, buttons, text boxes, etc., which are collectively called components
- ❑ Components can be divided into top-level containers, intermediate containers, and basic components according to different functions.
- ❑ Components and containers form a hierarchical relationship of inclusion.



# Swing frame





# Swing frame

---

- ❑ Swing's components are inherited from the JComponent class.
- ❑ The JComponent class provides the functionality that all components need.
- ❑ JComponent inherits from AWT's class Component and its subclass Container. Common components include the label JLabel, the key JButton, the input box JTextField, the checkbox JCheckBox, and the list JList.







# Swing frame

---

- ❑ A container is a special component that can contain components. There are two main types of containers in Swing.
- ❑ One category is **heavyweight containers, or top-level containers**, which do not inherit from Jcomponents, including JFrame, JApplet, and Jdialog. Their biggest feature is that they cannot be contained by other containers, but can only be used as the topmost container of the interface program to contain other components.
- ❑ The second type of containers are **lightweight containers, or Intermediate containers**, which are inherited from JComponent, including JPanel, JScrollPane, etc. Middle layer containers are used to hold several related components together. Since middle-tier containers inherit from JComponent, they are components themselves, and they can (and must) be included in other containers.



# Swing frame

---

- The **layout manager** controls the location of components in the container. When adding components to a container, you need to set up a layout manager for the container to manage the position of each component in the container, that is, the layout method.
- If you don't use the layout manager, you'll need to draw the positions of each component and calculate the distance between them before adding them to the container.



# Swing frame

---

- Java provides several layout managers.

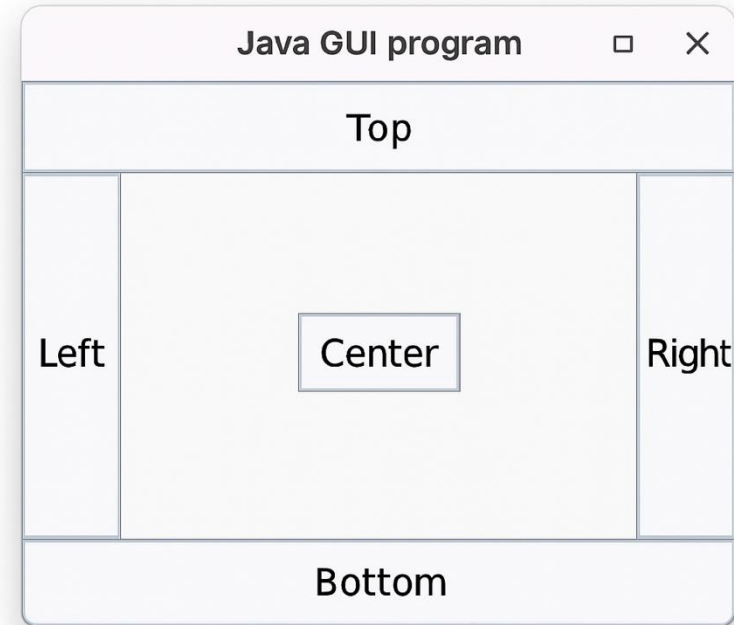
| Layout Manager | Characteristic                                                                                                              |
|----------------|-----------------------------------------------------------------------------------------------------------------------------|
| FlowLayout     | The streaming layout manager is placed from left to right, placed in the middle, and one row cannot be placed in the other. |
| BorderLayout   | The Border Layout Manager is divided into five directions: East, South, West, North, and Center.                            |
| GridLayout     | Mesh layout.                                                                                                                |
| GridBagLayout  | Mesh format layout allows for the placement of components of different sizes.                                               |
| BoxLayout      | Box layout manager to arrange components together horizontally or vertically.                                               |
| SpringLayout   | Organize components according to certain constraints.                                                                       |



# Swing frame

- Example: BorderLayout

```
JFrame frame=new JFrame("Java GUI program"); //Create a Frame window
frame.setSize(400,200);
frame.setLayout(new BorderLayout()); //Set the layout for the Frame window as BorderLayout
JButton button1=new JButton("Top");
JButton button2=new JButton("Left");
JButton button3=new JButton("Center");
JButton button4=new JButton("Right");
JButton button5=new JButton("Bottom");
frame.add(button1, BorderLayout.NORTH);
frame.add(button2, BorderLayout.WEST);
frame.add(button3, BorderLayout.CENTER);
frame.add(button4, BorderLayout.EAST);
frame.add(button5, BorderLayout.SOUTH);
frame.setBounds(300,200,600,300);
frame.setVisible(true);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```





## Content

- Swing frame
- **The principle of Swing graphic processing and drawing color**
- Event mechanism
- Swing basic user components
- MVC pattern

# Display frame

---

- ❑ The top-level window (the window that is not included in the other windows) is called a Frame.
- ❑ The class used to describe the top-level window in Swing is called JFrame, which extends the Frame library in AWT.
- ❑ JFrame is one of the very few Swing components that doesn't draw on the canvas. Its decorative parts (buttons, title bars, icons, etc.) are drawn by the user's window system.



# Display frame

---

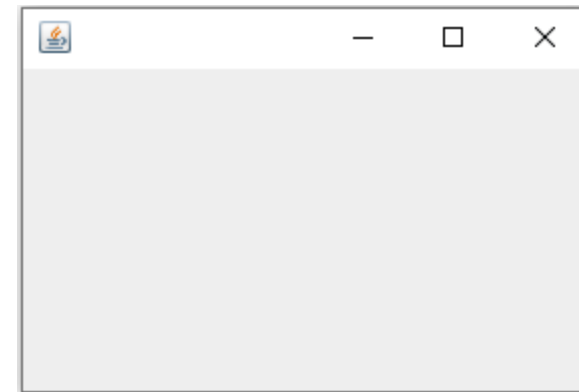
```
1. class SimpleFrame extends JFrame{
2.     private static final int DEFAULT_WIDTH = 300;
3.     private static final int DEFAULT_HEIGHT = 200;
4.
5.     public SimpleFrame(){
6.         setSize(DEFAULT_WIDTH,DEFAULT_HEIGHT);
7.     }
```

- By default, the form size is  $0 \times 0$  pixels.



# Display frame

```
1. public class SimpleFrameTest {  
2.     public static void main(String[] args){  
3.         EventQueue.invokeLater(()->  
4.             {  
5.                 var frame = new SimpleFrame();  
6.                 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
7.                 frame.setVisible(true);  
8.             });  
9.     }  
10. }
```



# Display frame

---

❑ `EventQueue.invokeLater()->{statements});`

- All Swing components must be configured by the **event dispatch thread**, which is the control thread that passes events such as mouse clicks and keystrokes to the user interface component.

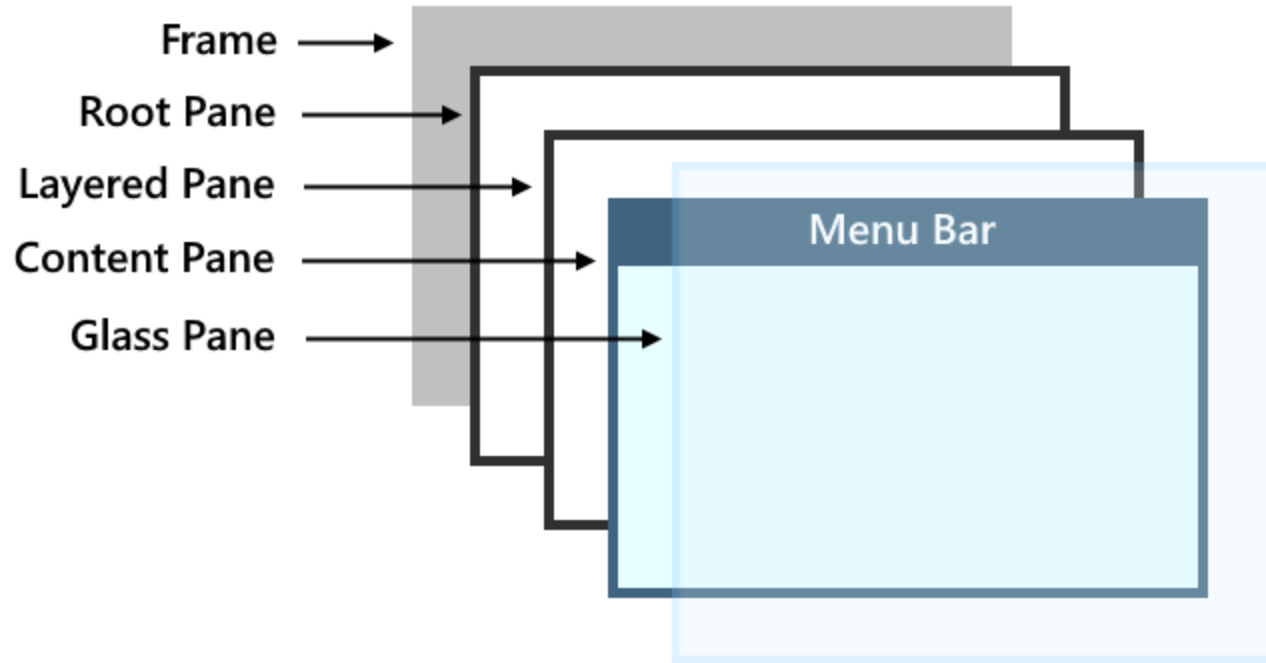
❑ `frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);`

- Define the corresponding action when the user closes the form—exit.

❑ `frame.setVisible(true);`

- The form is not visible at first. Programmers can add a series of components inward and make them display via the `setVisible` method.

# Display frame



- ❑ We just need to focus on the content pane.
- ❑ All components added to the form are automatically added to the content pane.

# Displays the information in the frame

---

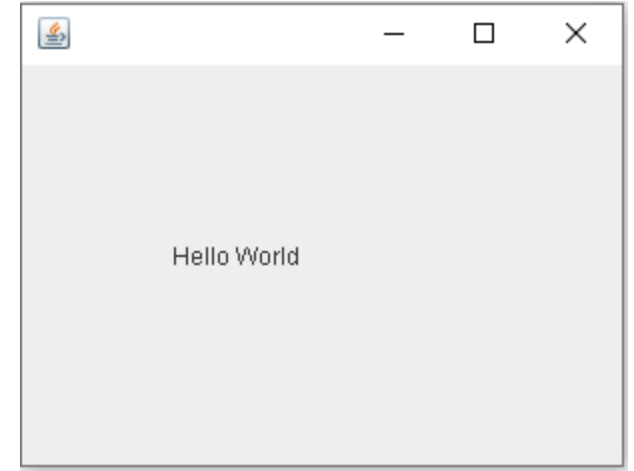
- ❑ We want to display information in a frame (e.g. the string "Hello World").
- ❑ We add a component to the frame, and the message will be drawn on this component.
- ❑ Drawing on a component:
  - Define a class that extends JComponent.
  - override the paintComponent method in it.
  - In the paintComponent method, set the Graphics object. The Graphics object contains methods for drawing patterns, images, and text. In Java, all drawing must be done through the Graphics object.

## Displays the information in the frame

```
1.class HelloWorldComponent extends JComponent{
2.    private static final int MSG_X = 75;
3.    private static final int MSG_Y = 100;
4.
5.    private static final int DEFAULT_WIDTH = 300;
6.    private static final int DEFAULT_HEIGHT = 200;
7.
8.    public void paintComponent(Graphics g){
9.        g.drawString("Hello World", MSG_X, MSG_Y);
10.    }
11.
12.    public Dimension getPreferredSize(){
13.        return new Dimension(DEFAULT_WIDTH, DEFAULT_HEIGHT);
14.    }
15.}
```

## ❑ Displays the information in the frame

```
13.class helloworldFrame extends JFrame{
14.
15.     public helloworldFrame()
16.     {
17.         add(new helloworldComponent());
18.         pack();
19.     }
19. }
```



- ❑ In Line 10, we set the size of the component to return a Dimension class object with a preferred width and height.
- ❑ Line 17, use the pack() method to use their preferred size when populating a frame.

# Draw 2D graphics

---

- ❑ Get an object of the Graphics2D class and draw the graph using the Java 2D library.
- ❑ The Java 2D library organizes geometry (lines, rectangles, ellipses) in an object-oriented manner:
  - Line2D
  - Rectangle2D
  - Ellipse2D
- ❑ The Java 2D library uses floating-point coordinates for pixels. Single-precision floats are used for internal calculations, and in order to avoid the processing of forced type conversion, you can use the Double graph class.

# Draw 2D graphics

---

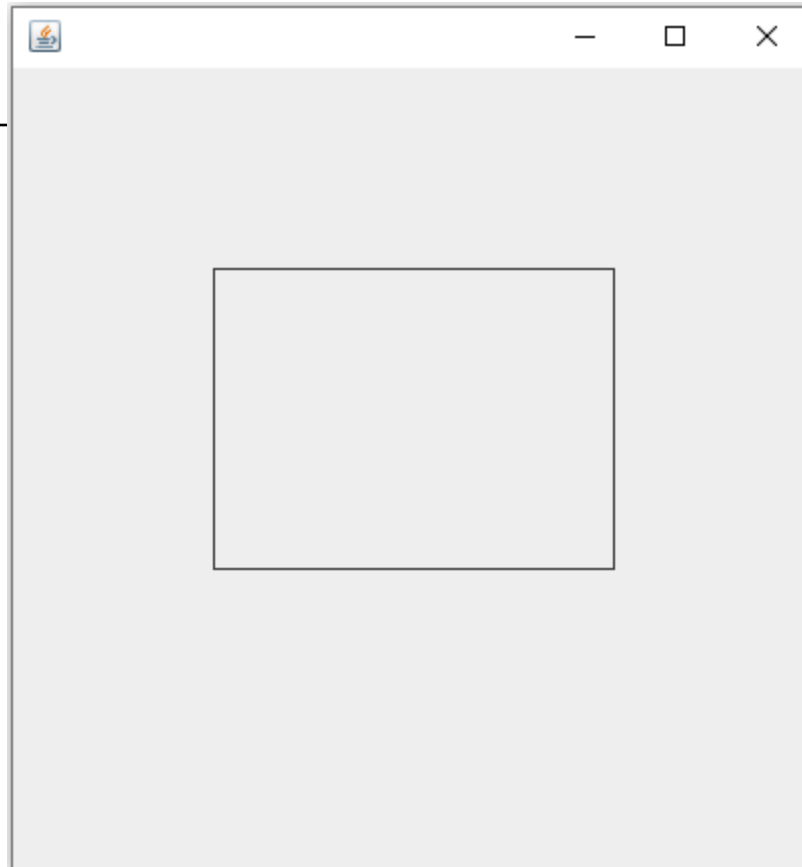
```
1. class DrawComponent extends JComponent{
2.     public static final int DEFAULT_WIDTH = 400;
3.     public static final int DEFAULT_HEIGHT = 400;
4.
5.     public void paintComponent(Graphics g){
6.         var g2 = (Graphics2D) g;
7.
8.         double leftX = 100;
9.         double topY = 100;
10.        double width = 200;
11.        double height = 150;
12.
13.        var rect = new Rectangle2D.Double(leftX, topY, width, height);
14.        g2.draw(rect);
15.    }
```



# □ Draw 2D graphics

---

```
14.    public Dimension getPreferredSize(){  
15.        return new Dimension(DEFAULT_WIDTH,DEFAULT_HEIGHT);  
16.    }  
17.  
18.}
```



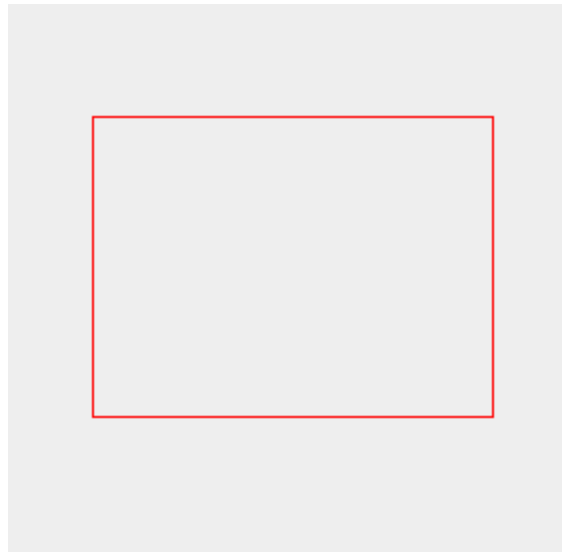


## Use color

---

- Use the `setPaint` method of the `Graphics2D` class to select colors for all subsequent draw operations in the context of the graphics.

```
var rect = new Rectangle2D.Double(leftX,topY,width,height);  
g2.setPaint(Color.RED);  
g2.draw(rect);
```





# Use color

---

- The interior of a closed graphic can be filled with one color.

```
var rect = new Rectangle2D.Double(leftX,topY,width,height);  
g2.setPaint(Color.RED);  
g2.fill(rect);
```



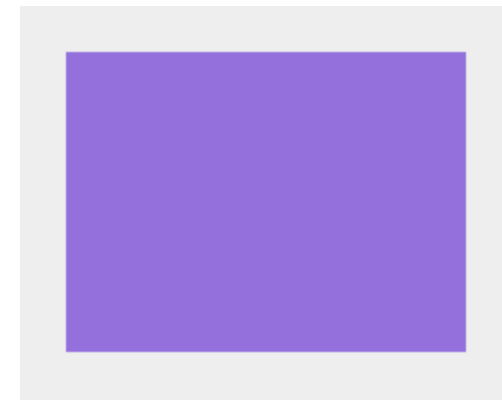


# Use color

---

- ❑ If you want to use multiple colors, you need to choose one color, draw a figure, and then choose another color and draw a figure.
- ❑ The Color class is used to define colors. The java.awt.Color class provides 13 predefined constants representing 13 standard colors.
- ❑ Tricolor components can be provided to create Color objects. The value is an integer between 0~255.

```
g2.setPaint(new Color(147,112,219));
```



# Show image

---

- ❑ You can use the ImageIcon class to read images from a file.
- ❑ The variable image contains a reference to an object that encapsulates the image data.

This image can be displayed using the drawImage method of the Graphics class.

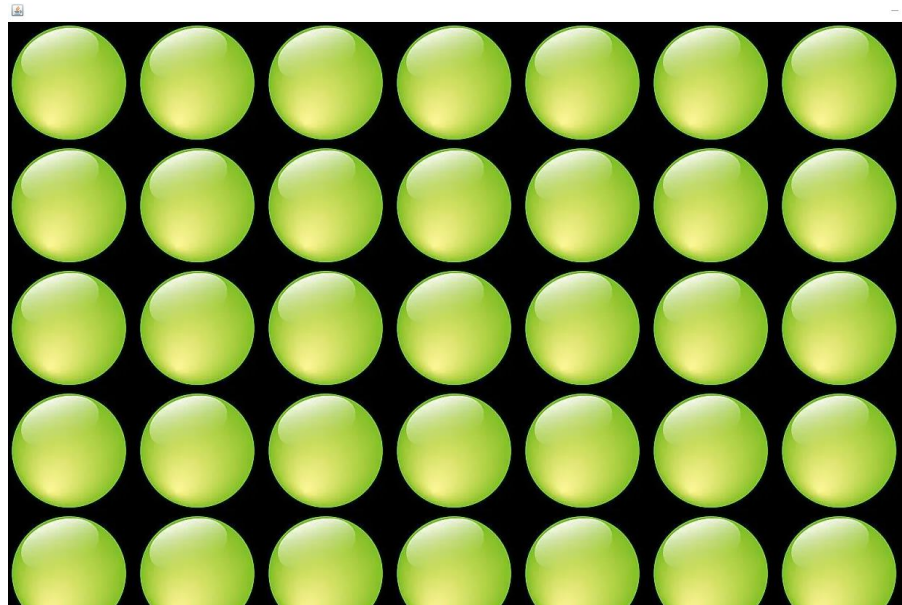
```
1.    public void paintComponent(Graphics g)
2.    {
3.        int X = 100;
4.        int Y = 100;
5.        Image image = new ImageIcon(path).getImage();
6.
7.        g.drawImage(image, X, Y, null);
8.    }
```



# Show image

---

- ❑ Take it a step further and til the image in a window.
- ❑ Start by displaying a copy of the image in the top left corner, then use the `copyArea` call to copy it to the entire window.







## Content

- Swing frame
- The principle of Swing graphic processing and drawing color
- **Event mechanism**
- Swing basic user components
- MVC pattern





# Event

---

## □ What is an event?

- Changing the state of an object is called an event
- Click a button, move the mouse, enter a character through the keyboard, and select an item from the list

# Event mechanism

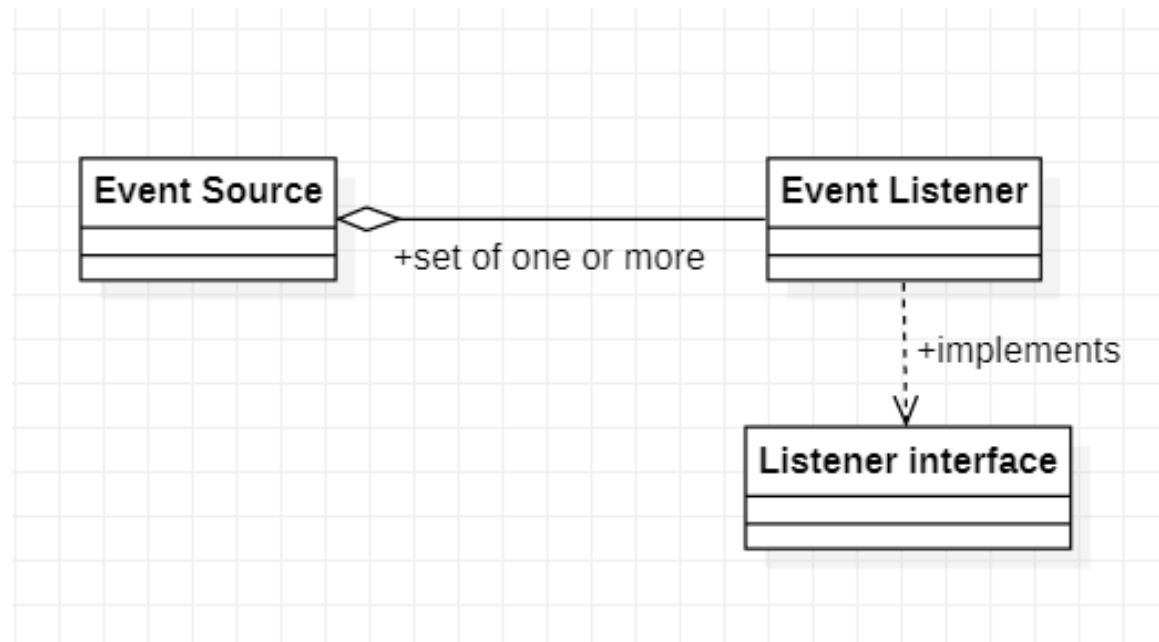
---

- ❑ Any operating environment that supports GUIs is constantly monitoring events such as keystrokes or mouse clicks. These events are then reported to the running program. Each program will decide how to respond to these events.
- ❑ Event Handling Mechanism (Three Types of Objects)
  - Event: An action by a user on a component is called an event
  - Event Source: The place where the event takes place, usually various components such as buttons or scroll bars.
  - Event Listener: A class instance that implements the listener interface.



# Event mechanism

- ❑ Event source objects are able to register and send event objects to listener objects.
- ❑ When an event occurs, the event source sends the event object to all registered listeners.
- ❑ The listener object then uses the information in the event object to determine how to respond to the event.



# Event mechanism

---

```
1.ActionListener listener = ... ;  
2.var button = new JButton("OK");  
3.button.addActionListener(listener)
```

- The listener object is notified whenever the button is clicked (which produces an "action event").



# Event mechanism

---

```
1. class MyListener implements ActionListener
2. {
3.     public void actionPerformed(ActionEvent event)
4.     {
5.         //reaction to button click goes here
6.     }
7. }
```

- ❑ When the user clicks the button, the JButton object creates an ActionEvent object.
- ❑ Then call listener.actionPerformed(event) and pass in this event object.
- ❑ An event source can have multiple listeners.



## Example: Click the button

---

```
1. public class ButtonFrame extends JFrame
2. {
3.     private JPanel buttonPanel;
4.     private static final int DEFAULT_WIDTH = 300;
5.     private static final int DEFAULT_HEIGHT = 200;
6.
7.     public ButtonFrame()
8.     {
9.         setSize(DEFAULT_WIDTH, DEFAULT_HEIGHT);
10.
11.         var YellowButton = new JButton("Yellow");
12.         var BlueButton = new JButton("Blue");
13.         var RedButton = new JButton("Red");
```



## Example: Click the button

---

```
13.         buttonPanel = new JPanel();

14.         buttonPanel.add(YellowButton);
15.         buttonPanel.add(BlueButton);
16.         buttonPanel.add(RedButton);

17.         add(buttonPanel);

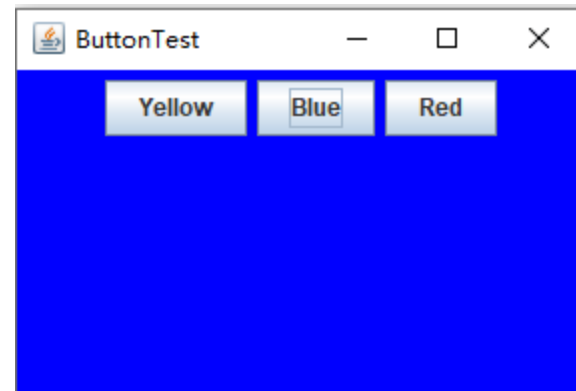
18.         var YellowAction = new ColorAction(Color.YELLOW);
19.         var BlueAction = new ColorAction(Color.BLUE);
20.         var RedAction = new ColorAction(Color.RED);

21.         YellowButton.addActionListener(YellowAction);
22.         BlueButton.addActionListener(BlueAction);
23.         RedButton.addActionListener(RedAction);
24.     }
```

## Example: Click the button

---

```
25. private class ColorAction implements ActionListener{
26.     private Color backgroundColor;
27.
28.     public ColorAction(Color C){
29.         backgroundColor = C;
30.     }
31.
32.     public void actionPerformed(ActionEvent event){
33.         buttonPanel.setBackground(backgroundColor);
34.     }
35. }
```





## Specifying the listener

---

- ❑ Typically, each listener performs a separate action.

```
exitButton.addActionListener(event->System.exit(0));
```

- ❑ There are multiple interrelated actions that can implement an auxiliary approach (take the color button as an example).

```
public void makeButton(String name, Color backgroundColor)
{
    var button = new JButton(name);
    buttonPanel.add(button);
    button.addActionListener(event->
        buttonPanel.setBackground(backgroundColor));
}
```

## Specifying the listener

---

- ❑ There are multiple interrelated actions that can implement an auxiliary approach that improves on the previous example.

```
public ButtonFrame2()  
{  
    setSize(DEFAULT_WIDTH, DEFAULT_HEIGHT);  
  
    buttonPanel = new JPanel();  
    add(buttonPanel);  
  
    makeButton("yellow", Color.YELLOW);  
    makeButton("blue", Color.BLUE);  
    makeButton("red", Color.RED);  
    makeButton("green", Color.GREEN);  
}
```





## Content

- Swing frame
- The principle of Swing graphic processing and drawing color
- Event mechanism
- **Swing basic user components**
- MVC pattern



# Text input

---

- ❑ Three methods inherited from the JTextComponent class (abstract class):
  - Text Field (JTextField): Receives a single line of text.
  - Text Area (JTextArea): Receives multiple lines of text.
  - Password Field (JPasswordField): Receives a single line of text and does not display the text content.
  
- ❑ Text domain
  - Change the content in the text field.
  - Get the text typed by the user.
  - trim(): Removes spaces before and after the content of the text field.



# Text input

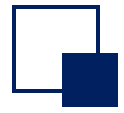
---

## □ Text area

- You can enter multiple lines of text, wrap the line with enter, each line ends with `\n`
- When constructing, you can specify the number of lines and columns in the text area

## □ Password field

- A special type of text field where each entered character is represented by an echo character
- A typical echo character: asterisk\*
- `char[] getPassword()`: The password is not returned as a String



# Text Input - Labels and Labels components

---

- ❑ Labels are the components that hold text.
- ❑ It does not respond to user input without any embellishments.
- ❑ Components can be identified using labels.
  - The text field itself is not identified.
  - Construct a JLabel component with the correct text.
  - Place it close enough to the component that needs to be identified.
- ❑ The JLabel constructor allows you to specify the initial text and icons, as well as how the content should be arranged.

```
new JLabel("User name:", SwingConstants.RIGHT)
```

## Text Input - Scroll pane

---

- ❑ In Swing, there is no scroll bar in the text area.
- ❑ If you need a scroll bar, you can place the text area in the scroll pane.

```
var textArea = new JTextArea(TEXTAREA_ROWS, TEXTAREA_COLUMNS);  
var scrollPane = new JScrollPane(textArea);
```

# Select the Component - checkbox

---

- ❑ The input received is only "yes" or "no"
  - ❑ Automatically with an identification label
  - ❑ An immediately adjacent tag is required to explain its purpose, specifying the label text in the constructor
  - ❑ Check/uncheck the box:
  - ❑ Get the current status of each checkbox:
- true: checked; false: Not checked



```
bold = new JCheckBox("Bold");
```

```
bold.setSelected(true);
```

```
bold.isSelected();
```



## Select the component - radio button

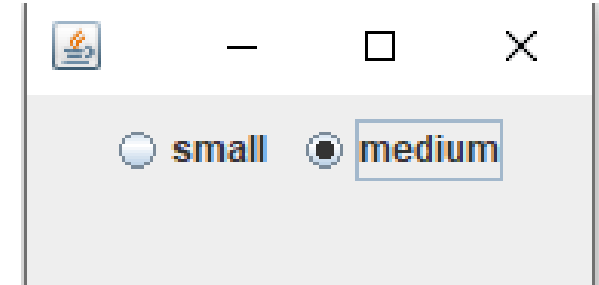
---

- ❑ Only one of the multiple selections can be selected.
- ❑ Construct an object of type ButtonGroup for the radio button group, and add an object of type JRadioButton to the button group.

```
var group = new ButtonGroup();  
  
var smallButton = new JRadioButton("small", false);  
//label small, initially unchecked  
group.add(smallButton);  
.....
```

## Select the component - radio button

- The ButtonGroup class has a getSelection method. You need to use the setActionCommand method to explicitly set action commands for all radio buttons.



ACTION Choice Selected: medium

```
var smallButton = new JRadioButton("small", false);  
smallButton.setActionCommand("small");           //Set action commands
```

```
class checkboxlistener implements ActionListener{  
    public void actionPerformed(ActionEvent ev) {  
        String choice = group.getSelection().getActionCommand();  
        System.out.println("ACTION Choice Selected: " + choice);  
        //Perform some functions  
    }  
}
```

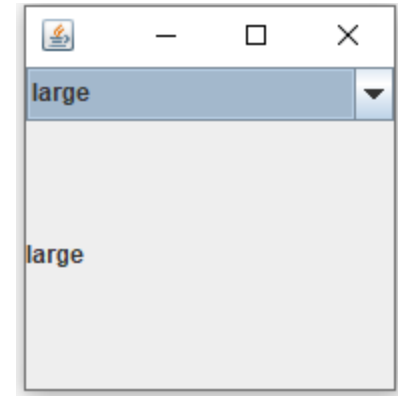
# Select the component - combo box

---

- ❑ Provides a drop-down selection list from which you can choose one.

- ❑ Construct the combo box and add options:

```
JComboBox<String> faceCombo = new JComboBox<>();  
faceCombo.addItem("small");  
faceCombo.addItem("medium");  
faceCombo.addItem("large");
```



- ❑ Listening Options:

```
faceCombo.addActionListener(event->  
label.setText(faceCombo.getSelectedItem().toString()));
```

- ❑ Delete options:

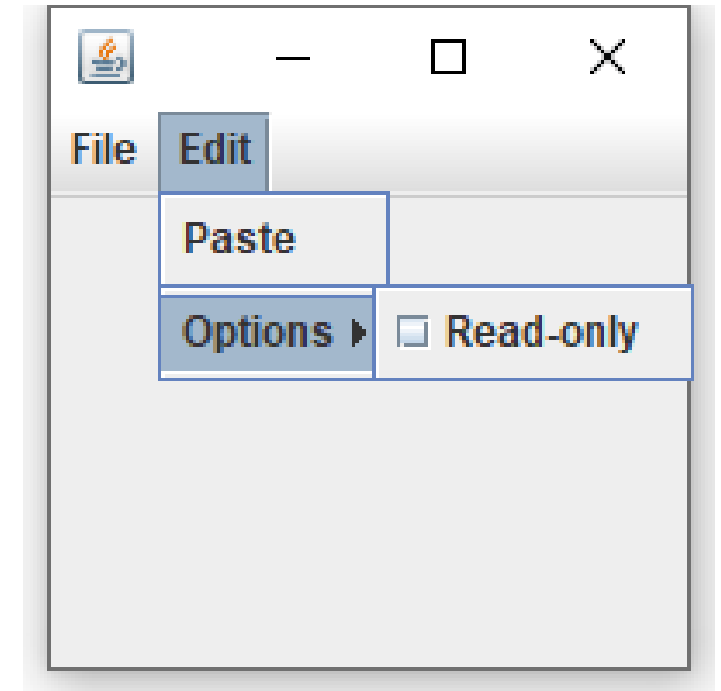
```
faceCombo.removeItem("small");  
faceCombo.removeItemAt(0);
```



# Menu

---

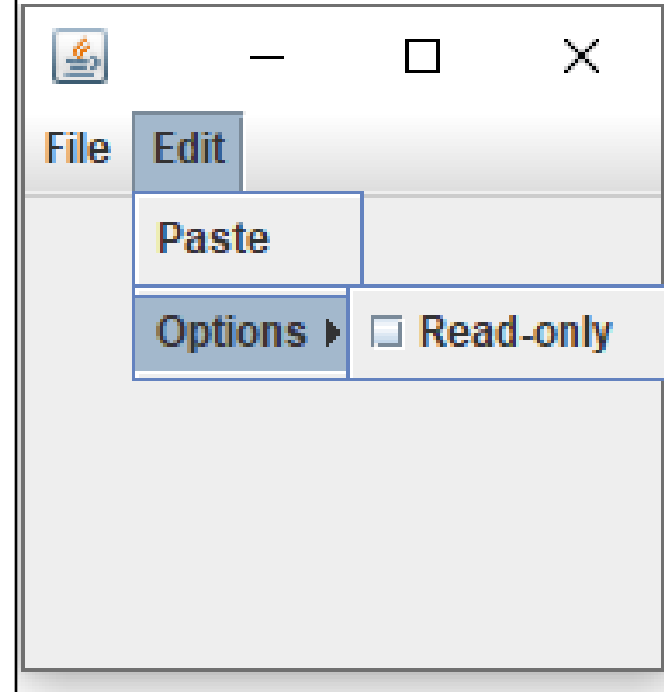
- ❑ The menu bar at the top of the window includes the name of the drop-down menu. Click on a name to open a menu with menu items and submenus.
- ❑ When the user clicks on a menu item, all menus are closed and a message is sent to the program.





# Menu

```
1. var menuBar = new JMenuBar();    //Create a menu bar
2. frame.setJMenuBar(menuBar);      //Place the menu bar in
   the frame
3. var fileMenu = new JMenu("File");
4. menuBar.add(fileMenu);
5. var editMenu = new JMenu("Edit");
6. menuBar.add(editMenu);
7. var pasteItem = new JMenuItem("Paste");
8. editMenu.add(pasteItem);
9. editMenu.addSeparator();
10. var readonlyItem = new JCheckBoxMenuItem("Read-only");
11. var optionMenu = new JMenu("Options");
12. optionMenu.add(readonlyItem);
13. editMenu.add(optionMenu);
```





# Content

- Swing frame
- The principle of Swing graphic processing and drawing color
- Event mechanism
- Swing basic user components
- **MVC pattern**



# MVC pattern

---

## □ Each component has three characteristics

- Content: whether the button is pressed; text in the text field, etc
- Appearance: color, size, etc
- Behavior: Reaction to an event

## □ To achieve these requirements, Swing uses a design pattern: model-view-controller (MVC)

- Model: Stores content
- View: Displays the content
- Controller: Handles user input

# MVC pattern

---

## □ Model:

- Store complete content.
- Implement ways to change content and find content.
- The model has no user interface and is completely invisible.

## □ View:

- A model can have multiple views.
- Each view can show different parts or different aspects of the entire content.
- When a model is updated, all views associated with it need to be notified to update synchronously.



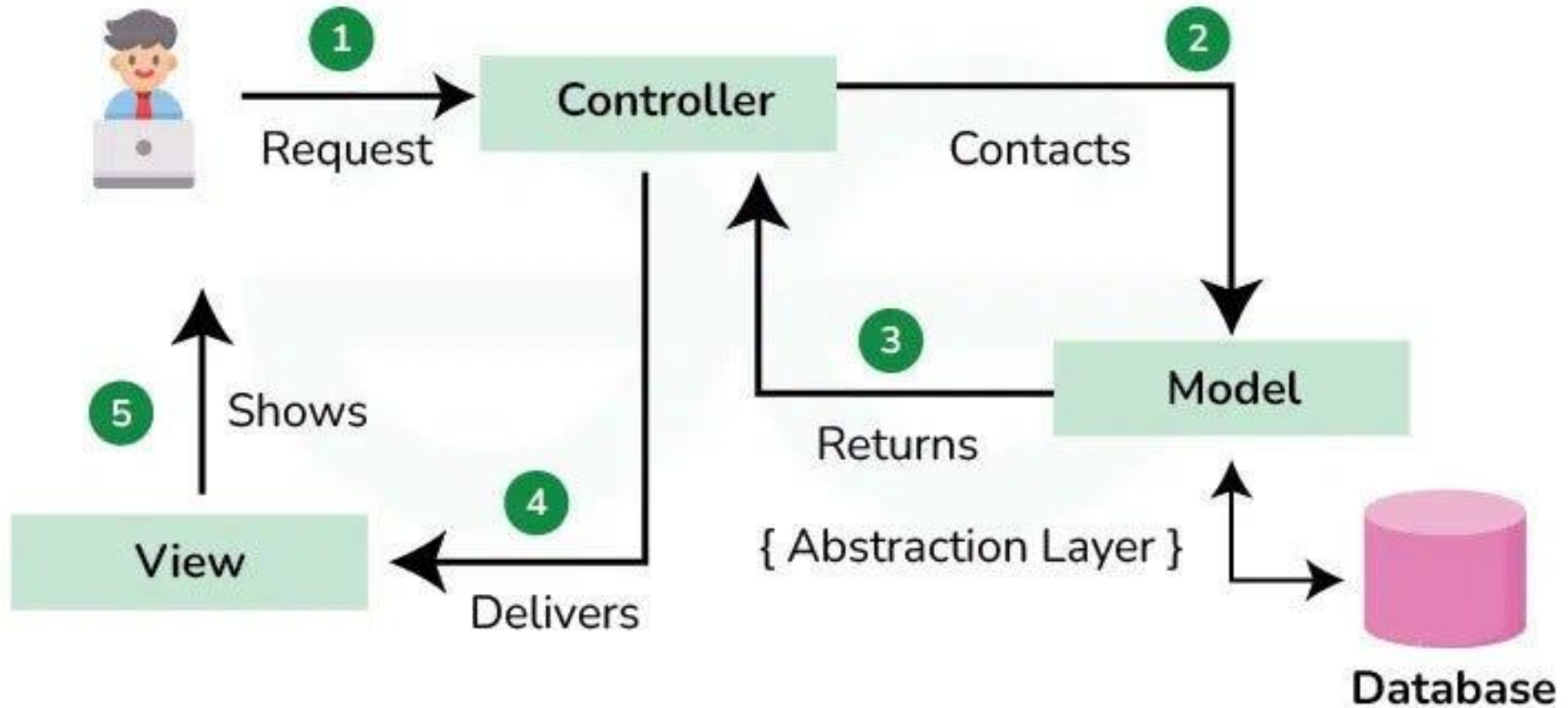
# MVC pattern

---

## □ Controller:

- Separate the view from the model
- Responsible for handling user input events such as mouse clicks and keystrokes.
- Decide whether to turn an event into a change to the model or view.
- Example: A user presses a key in a text box, the controller invokes the model's "insert character" command, and the model tells the view to update.
- Example: When a user presses an arrow key, the controller notifies the view to scroll, which has no effect on the underlying text.

# MVC pattern





# Question

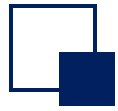
---

Create a small Swing app that shows a number and a button. When the button is clicked, the number increases by 1.

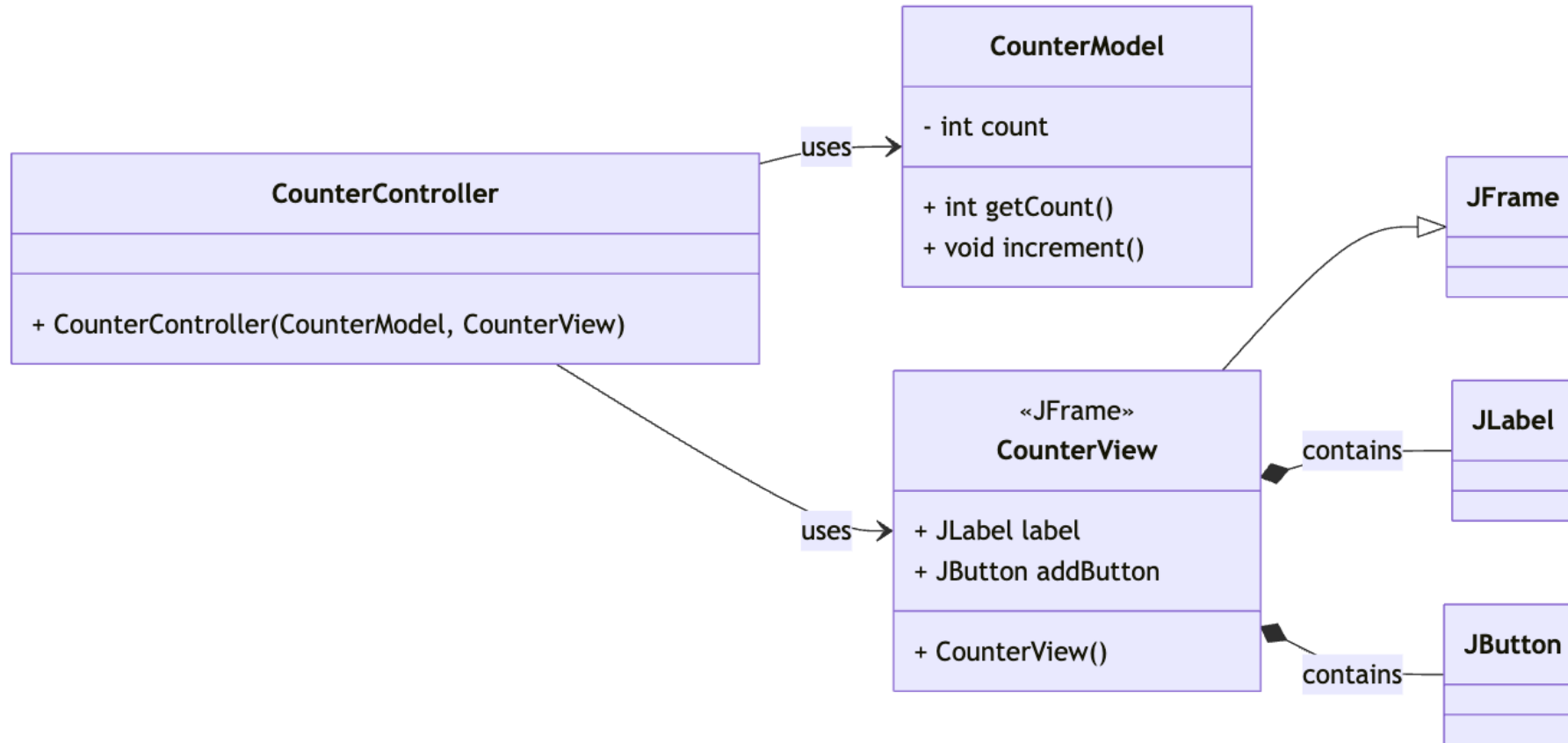
1. Design class diagram;
2. Summarize the responsibility for each class;

Hint:

1. Model (CounterModel): holds an int count, exposes getCount() and increment().
2. View (CounterView): a JFrame with a centered JLabel and a JButton at the bottom; provides getters to access those components.
3. Controller (CounterController): listens to the button, updates the model, then refreshes the label text.



# Question





# Question

```
import javax.swing.*;
import java.awt.event.*;

// Model: stores data
class CounterModel {
    private int count = 0;
    public int getCount() { return count; }
    public void increment() { count++; }
}

// View: shows UI
class CounterView extends JFrame {
    JButton addButton = new JButton("Add 1");
    JLabel label = new JLabel("0", SwingConstants.CENTER);
    CounterView() {
        setTitle("Simple MVC");
        setSize(200, 150);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        add(label, "Center");
        add(addButton, "South");
        setVisible(true);
    }
}
```

```
// Controller: handles events
class CounterController {
    CounterController(CounterModel m, CounterView v) {
        v.addButton.addActionListener(e -> {
            m.increment();
            v.label.setText(String.valueOf(m.getCount()));
        });
    }
}

public class SimpleMVC {
    public static void main(String[] args) {
        CounterModel m = new CounterModel();
        CounterView v = new CounterView();
        new CounterController(m, v);
    }
}
```



## Content

- **Swing frame**
- **The principle of Swing graphic processing and drawing color**
- **Event mechanism**
- **Swing basic user components**
- **MVC pattern**